

PrepHub Global Engineering Requirements

Status: Ready for Implementation

Version: 1.0

Applies To: All PrepHub PRDs and application features

1. Purpose

This document defines project-wide engineering requirements that apply across PrepHub.

Feature-specific PRDs define product behavior. This document defines shared standards for:

- Security
- Authorization
- Student-data handling
- Configuration
- Reliability
- Error handling
- Operational logging
- Backups
- Performance
- Accessibility
- Testing
- Deployment

When a feature PRD conflicts with this document, the more restrictive security or data-protection requirement takes precedence unless the conflict is deliberately resolved.

2. Authorization and Access Control

All restricted actions must be authorized server-side.

Hiding a button or route in the user interface is not sufficient authorization.

Every protected request must confirm:

- The user is authenticated.
- The account is active.
- The account has the required role.
- The user is permitted to access the requested resource.
- School Administrators are assigned to the relevant organization.
- Students own the requested student resource.

Required roles include:

OWNER
SCHOOL_ADMINISTRATOR
STUDENT

Owner and School Administrator routes must perform role and scope checks on every request.

3. Authentication Security

Passwords must be handled by the approved authentication system.

PrepHub must never:

- Store plaintext passwords
- Display existing passwords
- Log passwords
- Send passwords through email
- Expose password hashes

Password resets must use expiring, single-use reset links or the authentication provider's equivalent secure workflow.

Rate limiting

The following endpoints must use lightweight rate limiting:

- Login
- Password-reset requests
- School-email verification requests
- Account creation
- Other endpoints vulnerable to automated abuse

Rate limiting must:

- Avoid blocking ordinary student usage
- Return a safe error without revealing whether an account exists
- Support temporary restrictions rather than permanent lockouts
- Be configurable

Multi-factor authentication is not required in V1.

Recent-authentication confirmation for profile changes is not required in V1, but the application must still require an authenticated session and normal server-side authorization.

4. Data Minimization and Student Privacy

PrepHub must collect and retain only information required by its PRDs and legal or contractual obligations.

The application must not add behavioral tracking merely because the data could be useful later.

Student data may include only approved information such as:

- Account information
- Grade and graduation year
- School membership
- Diagnostic results
- Question attempts
- Ability Scores
- Predicted score history
- Practice progress
- Subscription status
- Required operational records

PrepHub must not collect unnecessary:

- Precise location
- Contacts
- Device files
- Advertising identifiers
- Cross-site browsing behavior
- Social-media activity
- Biometric data

No third-party behavioral analytics provider is required.

Any future third-party data processor must be separately reviewed before student data is transmitted to it.

5. Sensitive Data Handling

Application logs must never contain:

- Passwords
- Password-reset tokens
- Authentication tokens
- Session cookies
- Full payment-card data
- Stripe secrets
- Google OAuth secrets
- Raw numeric-response answers unless specifically required for debugging
- Complete request bodies containing sensitive information

User email addresses may appear only in restricted support or security logs when identifying the affected account is necessary.

General operational logs should use internal account IDs.

Logs containing email addresses must:

- Be accessible only to the Owner or authorized technical personnel
- Have limited retention
- Not be sent to public or broadly accessible logging systems
- Avoid including unrelated student academic data in the same log entry

Authorized support tools may resolve an internal account ID to an email address.

6. School Administrator Data Boundaries

School Administrators may access only:

- Organizations assigned to their administrator account
- Students actively or historically associated with those organizations
- Data permitted by PRD-011 and PRD-017
- Student activity occurring during the permitted membership period

School Administrators must not access:

- Students outside their assigned scope
- Individual payment information
- Stripe data
- Owner-only notes
- Student activity occurring after school access ended
- Other organizations' aggregate or individual data

These restrictions must be enforced server-side.

7. Account Deletion and Data Retention

Account deletion must remove or anonymize student data that is no longer required.

PrepHub may preserve records necessary for:

- Financial and tax obligations
- Payment disputes
- Security investigations
- Fraud prevention
- Historical referential integrity
- Anonymized aggregate reporting

Deleted accounts must not remain directly identifiable in ordinary product views.

Exact retention periods must remain consistent with:

- PrepHub's Privacy Policy
- Terms of Service
- School contracts
- Applicable legal requirements

Account deletion must be atomic and resistant to duplicate requests.

8. Application Environments

PrepHub does not require a permanent dedicated staging environment in V1.

It must still maintain separation between:

Production

Contains real student accounts, school data, subscriptions, and production services.

Non-production development or preview

Used to develop and test changes without modifying real student data.

This may consist of:

- Local development
- Temporary preview deployments
- Disposable test databases
- Provider sandbox or test modes

Production data must not be copied into development environments unless it has been properly anonymized.

9. External Service Separation

Non-production work must not use production credentials for:

- Stripe payments
- Email delivery
- Google authentication
- Object storage
- Databases
- Background jobs
- Webhooks

Stripe testing must use Stripe test mode.

Test email delivery must not send messages to real students unless explicitly intended and verified.

Production secrets must be stored in protected environment configuration and must never be committed to source control.

10. Centralized Configuration

The following must use centralized configuration rather than scattered hardcoded values:

- Adaptive-engine constants

- Predicted-score-engine thresholds
- File upload limits
- Rate limits
- Email timing
- Contract expiration timing
- Retry limits
- Feature flags
- External-service settings

Versioned algorithms must persist the configuration or engine version used for each relevant result.

Changing configuration must not retroactively alter:

- Completed attempts
- Generated practice sets
- Historical Ability Score updates
- Historical predicted-score records
- Active content revisions

11. Loading, Empty, Success, and Error States

Every user-facing page or significant component must define:

- Loading state
- Empty state
- Success state
- Recoverable error state
- Unrecoverable error state where applicable

The application must not display a blank screen while waiting for data.

Error messages must:

- Explain what the user can do next
- Avoid exposing stack traces or internal implementation details
- Provide a Retry action when retrying is safe
- Preserve entered data where practical

12. Optional Side-Effect Failures

A successful core action must remain successful if a nonessential side effect fails afterward.

Examples:

- An answer remains finalized if an optional email fails.
- A subscription remains valid if an internal notification fails.
- A school membership remains activated if a confirmation email fails.
- A question remains published if optional logging fails.

Failures of optional side effects must be logged and retried where appropriate.

Core transactional failures must still prevent incomplete state changes.

13. Idempotency and Atomicity

Consequential operations must be atomic and idempotent.

This includes:

- Answer finalization
- Diagnostic completion
- Practice-set completion
- Ability Score updates
- Prediction creation
- Stripe webhook processing
- Subscription activation and cancellation
- School-access activation
- Membership removal and restoration
- Content publishing and republishing
- Account deletion

A repeated request must not:

- Apply an Ability Score update twice
- Create duplicate subscriptions
- Create duplicate memberships
- Complete the same set twice
- Publish duplicate revisions
- Send duplicate consequential records

If an atomic operation fails, partial state must not be committed.

14. Session and Progress Persistence

PrepHub must preserve active student work after:

- Browser refresh
- Browser closure
- Temporary loss of internet connection
- Signing out and returning later
- Switching devices after saved data synchronizes

This applies to:

- Diagnostic assessments
- Adaptive Practice Sets
- Draft question answers
- Finalized question answers
- Current question position
- Session review state where applicable

An unfinished diagnostic or Practice Set must resume rather than regenerate.

15. Destructive Actions

Destructive or access-changing actions must require confirmation.

Examples include:

- Account deletion
- Content archiving
- Permanent content deletion
- Question unpublishing
- Organization suspension
- Student removal
- Subscription cancellation

Confirmation must explain:

- What will happen
 - When it takes effect
 - Whether data is preserved
 - Whether the action is reversible
-

16. Operational Logging

PrepHub must log meaningful operational and security failures.

Required categories include:

- Authentication failures
- Unauthorized-access attempts
- Practice-set generation failures
- Prediction-generation failures
- Payment and webhook failures
- School-email verification failures
- Media upload and processing failures
- Email delivery failures
- Unexpected server errors
- Backup failures
- Database connectivity failures

Logs must include enough context to investigate the issue, such as:

timestamp
environment
errorType
requestId
internalAccountId or restricted email
organizationId when applicable
affectedResourceId
safe error message

Repeated identical failures should be grouped or deduplicated where practical.

A dedicated Owner system-status page is not required in V1. Backend logs are sufficient.

17. Backups and Recovery

The production database must receive automated backups.

Required behavior:

- Daily backups at minimum
- Point-in-time recovery when supported by the database provider
- Backup encryption

- Restricted backup access
- Retention appropriate for operational recovery
- Monitoring for failed backups

Backup restoration must be tested periodically.

A backup is not considered reliable solely because the backup job reports success.

Question images and videos must use durable managed object storage rather than relying only on the application server's filesystem.

18. Published Content Preservation

Deleting, replacing, republishing, or unpublishing content must preserve revisions needed by:

- Active diagnostics
- Active Practice Sets
- Completed sessions
- Historical attempts
- Prediction or adaptive-engine audit records

A student's existing set must continue rendering the content version selected when the set was generated.

19. Performance Requirements

Under expected V1 traffic:

- Normal dashboard and list pages should become usable within approximately two seconds.
- Question navigation should feel immediate after a set loads.
- Current and nearby questions should be preloaded where practical.
- Practice-set generation must display a visible loading state.
- Long-running media uploads must display progress.
- Large tables should use server-side filtering, sorting, and pagination where appropriate.

Performance requirements are targets, not reasons to remove required validation or authorization.

20. Accessibility

Core functionality must support keyboard and mouse interaction.

Keyboard users must be able to:

- Navigate interactive controls
- Complete forms
- Move between assessment questions
- Select answer options
- Open and close dialogs
- Confirm or cancel actions

The platform must provide:

- Visible keyboard focus
- Programmatic form labels
- Clear validation messages
- Logical heading structure
- Descriptive button labels
- Alternative text for meaningful images
- Sufficient color contrast

Color must never be the only indication of:

- Correctness
- Errors
- Warnings
- Publishing status
- Membership status
- Progress changes

Core student and administrator workflows should target WCAG 2.1 AA.

Owner content authoring may remain desktop-first, but student workflows must remain usable at common laptop, tablet, and mobile widths.

21. Automated Testing

Every major algorithm must have automated unit tests based on its PRD examples and acceptance criteria.

Required algorithm coverage includes:

- Diagnostic initialization
- Ability Score updates
- Category Priority calculations
- Practice-set allocation
- Difficulty selection boundaries
- Question-selection tiers
- Predicted-score calculations
- School entitlement rules
- Billing status transitions

Critical end-to-end tests must cover:

- Account signup and login
- Diagnostic completion
- Adaptive Practice Set completion
- Individual subscription purchase
- School-access activation
- Owner question publishing

22. Database Migrations

All database schema changes must use version-controlled migrations.

A permanent staging environment is not required.

Before production deployment, a migration must still be tested against:

- A disposable non-production database
- A local database matching the production engine
- Or an isolated preview database

Migration testing must confirm:

- The migration completes successfully.
- Existing representative records remain valid.
- Required indexes and constraints are created.
- Rollback or recovery steps are known.
- Production secrets and real student data are not exposed.

Schema changes must never be tested for the first time against the live production database.

23. Deployment and Rollback

Production deployments must support rollback or rapid recovery when a release causes serious failures.

Before deployment:

- Automated tests must pass.
- Required migrations must be validated.
- Production configuration must be checked.
- Secrets must be present.
- Critical dependencies must be available.

After deployment:

- Basic authentication must be verified.
 - Core student pages must load.
 - Existing sessions must remain accessible.
 - Practice and billing systems must be checked where affected.
-

24. Completion Standard

A feature is complete only when:

- Its PRD requirements are implemented.
 - Its acceptance criteria pass.
 - Required authorization is enforced.
 - Loading and error states exist.
 - Required automated tests pass.
 - Data migrations are complete.
 - Relevant operational failures are logged.
 - No known critical security or data-integrity issue remains.
-

25. Core Invariants

The following must always remain true:

- Restricted actions are authorized server-side.
- Production secrets never appear in source code.
- Production student data is not used casually for development.

- Optional logging failures do not break student actions.
- Consequential operations cannot execute twice.
- Active student work survives normal interruptions.
- Logs do not contain passwords, tokens, or payment-card data.
- Published historical content remains available to existing records.
- Database changes are tested outside production first.
- Core functionality supports keyboard and mouse input.
- A feature is not complete merely because its interface appears functional.

End of Global Engineering Requirements